

helixqa_integration_sec10

10. Phase 9: Build & Automation Integration

HelixQA must become an invocable target within HelixCode's existing build surface, not an external process that operators remember to run. The constraint is constitutional: the project mandates *no* CI/CD pipelines, *no* `.github/workflows/`, and *no* automated remote triggers. All quality automation is therefore expressed as Makefile targets and shell scripts that an operator executes manually on a workstation or build host. This chapter specifies the exact targets, their dependency chains, the scripts that implement them, and the resource-governance rules that keep the suite within the 30–40% host-resource ceiling defined by the constitution.

10.1 Makefile Targets

HelixCode's `Makefile` (15,341 characters, nested at `HelixCode/Makefile`) already exposes `build`, `test`, `test-integration`, `test-e2e`, and `test-challenges`. HelixQA integration appends five new top-level targets that compose the existing ones with QA-specific stages. Every target carries an explicit `QA_RESOURCE_CAP` annotation defaulting to 35%, the midpoint of the constitutional 30–40% band.

Target `qa-all` runs the complete pipeline: `unit` → `integration` → `e2e` → `screenshot-verify` → `anti-bluff` → `challenges` → `report`. A failure in an early stage halts the pipeline before later stages consume resources. The `timeout` utility enforces a 4-hour ceiling.

— HelixQA Integration Targets

```
.PHONY: qa-all qa-session qa-anti-bluff qa-screenshot qa-report
```

```
QA_TIMEOUT           ?= 4h
QA_RESOURCE_CAP      ?= 35
QA_OUTPUT_DIR        ?= ./qa-results
QA_BANKS_DIR         ?= ../HelixQA/banks
QA_CHALLENGE_DIR     ?= ../HelixQA/challenges/scripts
```

```
qa-all: build test test-integration test-e2e qa-screenshot qa-
    anti-bluff test-challenges qa-report
    @echo "[qa-all] Full QA pipeline completed. Report: $(
        QA_OUTPUT_DIR)/report.md"
    @if [ -f $(QA_OUTPUT_DIR)/report.json ]; then \
        jq '.failed_challenges' $(QA_OUTPUT_DIR)/report.json | \
        grep -qv '0' && exit 1 || exit 0; \
    fi
```

The final `if` block extracts `failed_challenges` from `report.json` and returns non-zero when failures are present, preserving Make's contract that a failed target aborts dependent work.

Target `qa-session` launches a full autonomous QA session. It accepts `QA_PLATFORMS` (default `android,desktop,web`) and `QA_COVERAGE_TARGET` (default `0.90`). The resource guard runs before the session starts.

`qa-session: build`

```
@echo "[qa-session] Resource guard: max $(QA_RESOURCE_CAP)%  
host usage"  
@bash scripts/resource_guard.sh $(QA_RESOURCE_CAP)  
@mkdir -p $(QA_OUTPUT_DIR)  
@timeout $(QA_TIMEOUT) ../HelixQA/bin/helixqa autonomous \  
--project $(PWD) \  
--platforms $(QA_PLATFORMS) \  
--env .env.qa \  
--timeout 2h \  
--output $(QA_OUTPUT_DIR)/session  
@echo "[qa-session] Artifacts: $(QA_OUTPUT_DIR)/session"
```

Target `qa-anti-bluff` executes the deliberate-break validation suite. It injects known defects into a temporary build tree, runs the corresponding tests, and asserts that those tests *fail*. A passing test under an injected defect is flagged as a bluff under `CONST-035` (HelixQA) / `CONST-017` (HelixCode), a release-blocking condition per the constitution.

`qa-anti-bluff: build`

```
@echo "[qa-anti-bluff] Running deliberate-break validation"  
@mkdir -p $(QA_OUTPUT_DIR)/anti-bluff  
@bash scripts/run_anti_bluff.sh \  
--source $(PWD) \  
--output $(QA_OUTPUT_DIR)/anti-bluff \  
--resource-cap $(QA_RESOURCE_CAP)  
@echo "[qa-anti-bluff] Complete: $(QA_OUTPUT_DIR)/anti-  
bluff/"
```

Target `qa-screenshot` captures platform screenshots on demand by launching each client application and storing images under `$(QA_OUTPUT_DIR)/screenshots/`. It is not part of the `qa-all` dependency graph; it is intended for ad-hoc visual regression baselines.

`qa-screenshot: build`

```
@echo "[qa-screenshot] Capturing screenshots for all client  
apps"  
@mkdir -p $(QA_OUTPUT_DIR)/screenshots  
@bash scripts/capture_screenshots.sh \  
--output $(QA_OUTPUT_DIR)/screenshots \  
--bin-dir ./bin \  
--resource-cap $(QA_RESOURCE_CAP)  
@echo "[qa-screenshot] Written: $(QA_OUTPUT_DIR)/  
screenshots/"
```

Target `qa-report` aggregates artifacts from earlier stages into Markdown, HTML, and JSON reports. It has no runtime dependency on the applications themselves.

```

qa-report:
    @echo "[qa-report] Generating consolidated QA report"
    @mkdir -p $(QA_OUTPUT_DIR)
    @../HelixQA/bin/helixqa report \
        --input $(QA_OUTPUT_DIR) \
        --format markdown,html,json \
        --output $(QA_OUTPUT_DIR)/report
    @echo "[qa-report] $(QA_OUTPUT_DIR)/report.{md,html,json}"

```

Target	Dependencies	Primary Command	Output	Resource Lin
qa-all	build test test-integration test-e2e qa-screenshot qa-anti-bluff test-challenges qa-report	Sequential pipeline with timeout 4h	qa-results/ report. {md,html,json}	35% host cap resource_
qa-session	build	helixqa autonomous -- platforms ... --timeout 2h	qa-results/ session/ with videos, screenshots, tickets	35% host cap before launch
qa-anti-bluff	build	run_anti_bluff.sh -- source ... --resource- cap 35	qa-results/ anti-bluff/ with bluff-detection log	35% host cap script
qa-screenshot	build	capture_screenshots.sh --bin-dir ./bin	qa-results/ screenshots/ *.png per platform	35% host cap script
qa-report	(aggregation only)	helixqa report --input qa-results/ --format ...	qa-results/ report. {md,html,json}	Negligible (n launch)

The dependency graph guarantees structural ordering: `qa-all` chains seven stages, and if `test-integration` fails, `qa-screenshot` and all later stages are skipped. The 35% default cap is surfaced as a tunable `QA_RESOURCE_CAP` so operators on smaller hosts can reduce it to 30% or raise it to 40% on dedicated workstations.

10.2 Session Scripts

The Makefile delegates to shell scripts in `HelixCode/scripts/`. These scripts implement timeout handling, process supervision, resource monitoring, screenshot validation, and multi-platform challenge orchestration. All scripts assume standard Unix utilities (`timeout`, `ulimit`, `jq`, `ffmpeg`, `adb` when Android testing is enabled).

10.2.1 scripts/run_full_qa.sh

This script is the reference implementation of the qa-all target's runtime logic. It can also be executed directly when an operator wants extra flags not exposed in the Makefile.

```
#!/usr/bin/env bash
set -euo pipefail

PROJECT_ROOT="$(cd "$(dirname "${BASH_SOURCE[0]}")/.." && pwd)"
QA_OUTPUT_DIR="${QA_OUTPUT_DIR:-${PROJECT_ROOT}/qa-results}"
QA_RESOURCE_CAP="${QA_RESOURCE_CAP:-35}"
QA_TIMEOUT="${QA_TIMEOUT:-4h}"
QA_PLATFORMS="${QA_PLATFORMS:-android,desktop,web}"

log() { echo "[run_full_qa] $(date '+%Y-%m-%d %H:%M:%S') $*"; }
die() { log "FATAL: $*"; exit 1; }

bash "${PROJECT_ROOT}/scripts/resource_guard.sh" "$QA_RESOURCE_CAP" || \
    die "Host resource check failed (cap=${QA_RESOURCE_CAP}%)"

mkdir -p "${QA_OUTPUT_DIR}"

log "Stage 1: Build"
cd "${PROJECT_ROOT}/HelixCode" && make build

log "Stage 2: Unit tests"
make test

log "Stage 3: Integration tests"
make test-integration

log "Stage 4: E2E tests"
make test-e2e

log "Stage 5: Screenshot capture"
mkdir -p "${QA_OUTPUT_DIR}/screenshots"
bash "${PROJECT_ROOT}/scripts/capture_screenshots.sh" \
    --output "${QA_OUTPUT_DIR}/screenshots" \
    --bin-dir "${PROJECT_ROOT}/HelixCode/bin" \
    --resource-cap "${QA_RESOURCE_CAP}"

log "Stage 6: Anti-bluff validation"
mkdir -p "${QA_OUTPUT_DIR}/anti-bluff"
bash "${PROJECT_ROOT}/scripts/run_anti_bluff.sh" \
    --source "${PROJECT_ROOT}/HelixCode" \
    --output "${QA_OUTPUT_DIR}/anti-bluff" \
    --resource-cap "${QA_RESOURCE_CAP}"
```

```
log "Stage 7: Challenges"
make test-challenges
```

```
log "Stage 8: Report generation"
"${PROJECT_ROOT}/HelixQA/bin/helixqa" report \
  --input "${QA_OUTPUT_DIR}" \
  --format markdown,html,json \
  --output "${QA_OUTPUT_DIR}/report"
```

```
log "Pipeline complete. Report: ${QA_OUTPUT_DIR}/report.md"
```

set -euo pipefail ensures any failed stage exits immediately. The timeout wrapper from the Makefile applies to the entire invocation; within the script, individual stages rely on the Go test runner's own timeouts (-timeout 30m for unit tests, -timeout 60m for e2e tests, configured in the existing Makefile).

10.2.2 scripts/run_nightly_qa.sh

This script is for scheduled heavy QA sessions, invoked via cron on a dedicated build workstation. The constitution prohibits CI/CD pipelines; cron on an operator-controlled host is the only permissible form of scheduled automation.

```
#!/usr/bin/env bash
set -euo pipefail

PROJECT_ROOT="$(cd "$(dirname "${BASH_SOURCE[0]}")/.." && pwd)"
QA_OUTPUT_DIR="${QA_OUTPUT_DIR:-${PROJECT_ROOT}/qa-results/nightly}"
QA_RESOURCE_CAP="${QA_RESOURCE_CAP:-40}"
QA_TIMEOUT="${QA_TIMEOUT:-8h}"
QA_PLATFORMS="$
    {QA_PLATFORMS:-android,androidtv,desktop,web,cli,api}"

echo "[nightly-qa] $(date) Starting on platforms: $
    {QA_PLATFORMS}"

bash "${PROJECT_ROOT}/scripts/resource_guard.sh" "$
    {QA_RESOURCE_CAP}" || {
    echo "[nightly-qa] Resource guard failed — aborting"; exit 1;
}

mkdir -p "${QA_OUTPUT_DIR}"
cd "${PROJECT_ROOT}/HelixCode" && make build

"${PROJECT_ROOT}/HelixQA/bin/helixqa" run \
  --banks "${PROJECT_ROOT}/HelixQA/banks" \
  --platform "${QA_PLATFORMS}" \
  --output "${QA_OUTPUT_DIR}" \
  --speed slow \
  --report markdown,html,json \
```

```

--validate --record \
--timeout "${QA_TIMEOUT}"

"${PROJECT_ROOT}/HelixQA/bin/helixqa" autonomous \
--project "${PROJECT_ROOT}/HelixCode" \
--platforms "${QA_PLATFORMS}" \
--env "${PROJECT_ROOT}/HelixCode/.env.full-test" \
--timeout "${QA_TIMEOUT}" \
--output "${QA_OUTPUT_DIR}/autonomous"

echo "[nightly-qa] $(date) Complete. Artifacts: ${QA_OUTPUT_DIR}"

```

The nightly script raises the resource cap to 40%, the top of the constitutional band, because it runs on a dedicated host. `--speed slow` reduces CPU contention during the orchestrated bank run; `--record` captures video evidence for every platform.

10.2.3 scripts/verify_screenshots.sh

Screenshot validation is a critical anti-bluff gate. A screenshot that is empty, uniformly colored, or missing expected UI elements is not valid evidence and must fail the gate.

```

#!/usr/bin/env bash
set -euo pipefail

SCREENSHOT_DIR=""
EXPECTED_ELEMENTS_FILE=""
FAIL_ON_INVALID=1

while [[ $# -gt 0 ]]; do
    case "$1" in
        --dir) SCREENSHOT_DIR="$2"; shift 2 ;;
        --expected-elements) EXPECTED_ELEMENTS_FILE="$2"; shift 2 ;;
        --warn-only) FAIL_ON_INVALID=0; shift ;;
        *) echo "Unknown arg: $1"; exit 1 ;;
    esac
done

[[ -d "${SCREENSHOT_DIR}" ]] || { echo "Directory not found: $
    {SCREENSHOT_DIR}"; exit 1; }

INVALID_COUNT=0

for img in "${SCREENSHOT_DIR}"/*.png; do
    [[ -f "${img}" ]] || continue
    SIZE=$(stat -c%s "${img}" 2>/dev/null || stat -f%z "${img}")

    if [[ "${SIZE}" -lt 100 ]]; then
        echo "EMPTY: ${img} (size=${SIZE})"
    fi
done

```

```

        ((INVALID_COUNT++)); continue
    fi

    if command -v convert >/dev/null 2>&1; then
        VARIANCE=$(convert "${img}" -format '%[standard-deviation]' info: 2>/dev/null || echo "0")
        if [[ "${VARIANCE%.*}" -eq 0 ]]; then
            echo "UNIFORM: ${img} (stddev=0)"
            ((INVALID_COUNT++)); continue
        fi
    fi

    if [[ -n "${EXPECTED_ELEMENTS_FILE}" && -f "${EXPECTED_ELEMENTS_FILE}" ]]; then
        BASENAME=$(basename "${img}")
        if ! grep -q "${BASENAME}" "${EXPECTED_ELEMENTS_FILE}"; then
            echo "MISSING_ELEMENTS: ${img}"
            ((INVALID_COUNT++)); continue
        fi
    fi

    echo "VALID: ${img}"
done

echo "verify_screenshots: ${INVALID_COUNT} invalid"
[[ "${FAIL_ON_INVALID}" -eq 1 && "${INVALID_COUNT}" -gt 0 ]] &&
    exit 1
exit 0

```

The `--warn-only` mode is available for initial pipeline integration; once baselines are established, the default behavior enforces CONST-035: a screenshot gate that passes on blank frames is itself a bluff.

10.2.4 scripts/run_all_challenges.sh

The existing 7-phase challenge suite is extended to include anti-bluff and screenshot challenges. The extension wraps the original runner and appends new validation stages.

```

#!/usr/bin/env bash
set -euo pipefail

PROJECT_ROOT="$(cd "$(dirname "${BASH_SOURCE[0]}")/.." && pwd)"
QA_OUTPUT_DIR="${QA_OUTPUT_DIR:-${PROJECT_ROOT}/qa-results/
    challenges}"
QA_RESOURCE_CAP="${QA_RESOURCE_CAP:-35}"

mkdir -p "${QA_OUTPUT_DIR}"

```

```

echo "[challenges] Phase 1-7: Base suite"
bash "${PROJECT_ROOT}/challenges/scripts/run_all_challenges.sh" \
    --output "${QA_OUTPUT_DIR}/base"

echo "[challenges] Phase 8: Anti-bluff"
bash "${PROJECT_ROOT}/scripts/run_anti_bluff.sh" \
    --source "${PROJECT_ROOT}/HelixCode" \
    --output "${QA_OUTPUT_DIR}/anti-bluff" \
    --resource-cap "${QA_RESOURCE_CAP}"

echo "[challenges] Phase 9: Screenshot verification"
bash "${PROJECT_ROOT}/scripts/verify_screenshots.sh" \
    --dir "${PROJECT_ROOT}/qa-results/screenshots" \
    --expected-elements "${PROJECT_ROOT}/scripts/
        expected_elements.txt"

echo "[challenges] All phases complete: ${QA_OUTPUT_DIR}"

```

The extended suite produces a unified exit code: any non-zero phase fails the entire run, satisfying CONST-035’s requirement that a green summary line with a broken feature is a critical defect.

Script	Purpose	Trigger	Platforms
scripts/ run_full_qa.sh	Complete QA pipeline with resource guard	make qa-all or manual	android,desktop,we
scripts/ run_nightly_qa.sh	Heavy scheduled session with full bank coverage	cron or systemd timer on dedicated host	android,androidtv,
scripts/ verify_screenshots.sh	Validate screenshots are non-empty, non-uniform, contain expected elements	make qa-screenshot or run_all_challenges.sh	All captured platforms
scripts/ run_all_challenges.sh	Extended 9-phase suite (base 7 + anti-bluff + screenshot)	make test-challenges or manual	All platforms in challenge c

The scripts are composable: `run_full_qa.sh` invokes `verify_screenshots.sh`; `run_nightly_qa.sh` invokes both `helixqa run` and `helixqa autonomous`. An operator who wants only the anti-bluff gate can execute `bash scripts/run_anti_bluff.sh` directly. This composability is the advantage of a script-based model over a CI/CD pipeline — each stage is independently inspectable, debuggable, and replaceable.

10.3 No-CI/CD Compliance

The HelixCode constitution prohibits CI/CD pipelines across `CONSTITUTION.md`, `CLAUDE.md`, and `AGENTS.md`, propagated to all submodules per CONST-036.

10.3.1 Absence of `.github/workflows/`

HelixCode contains a `.github/workflows/` directory at `HelixCode/.github/workflows/`, but the constitutional mandate requires that *no* CI/CD pipelines exist. The integration plan treats any existing workflow files as a pre-existing condition to be deprecated, not extended. All new automation is expressed exclusively in `Makefile` and `scripts/`; no GitHub Actions, GitLab CI, Jenkinsfile, Tekton pipeline, or remote webhook trigger is added.

10.3.2 Manual Trigger Model

Every QA operation is initiated by an operator running a Make target or script on a host they control. After a significant commit — for example, a provider integration change in `internal/llm/` or a UI rework in `applications/desktop/` — the operator runs `make qa-all` and waits for the pipeline to complete. There is no automated post-push trigger. Article XI §11.9 states that “the bar for shipping is not ‘tests pass’ but ‘users can use the feature’”; manual execution forces a human review step that automated pipelines bypass.

10.3.3 Pre-commit Hooks Prohibited; Local Verification via `scripts/pre-validate.sh`

The constitution prohibits pre-commit hooks because they impose unreviewed executable code into the Git lifecycle. In their place, a `scripts/pre-validate.sh` script provides an optional local gate that an operator can run before committing, but which does not execute automatically.

```
#!/usr/bin/env bash
set -euo pipefail

# pre-validate.sh — optional local gate, NOT a Git hook
PROJECT_ROOT="$(cd "$(dirname "${BASH_SOURCE[0]}")/.." && pwd)"

echo "[pre-validate] Local verification (no Git hook invoked)"
cd "${PROJECT_ROOT}/HelixCode" || exit 1
make lint
make test
make test-integration
echo "[pre-validate] Passed. You may now commit."
```

The header comment explicitly states that the script is *not* a Git hook, preserving constitutional compliance. Operators who want convenience can create a shell alias, but the repository itself installs no `.git/hooks/` content.

10.3.4 Build Orchestrator Owns Container Lifecycle

Docker containers are launched only through Make targets (`make docker-build`, `make docker-compose-up`) or through the `digital.vasic.containers` sibling module. Direct `docker` commands by operators are discouraged because they bypass the resource-limit enforcement and health-check configuration embedded in the orchestrator. QA targets that need a running stack depend on these Make targets rather than issuing ad-hoc `docker run` invocations. This ownership model ensures that container resource limits (memory caps, CPU quotas) are consistently applied and that the constitutional 30–40% ceiling is never exceeded by container sprawl.

The resource-guard script (`scripts/resource_guard.sh`) is the enforcement point. It reads `/proc/stat` and `/proc/meminfo` (Linux) or uses `vm_stat` and `top` (macOS) to compute current host utilization, then compares projected additional load against the cap. If the projected total exceeds the cap, the script exits with code 1 and the target aborts before any test starts. Without this guard, an operator running `make qa-all` on a shared workstation could exhaust host resources and trigger an out-of-memory kill — an event the constitution classifies as a host-power-management violation under CONST-033 (“Host Power Management is Forbidden”). By enforcing the cap at the entry point, the architecture prevents resource exhaustion before it occurs.